PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

实验三 模型机组合部件的实现（二）

班级 姓名 学号

一、实验目的

1．了解简易模型机的内部结构和工作原理。

2．分析模型机的功能，设计 8 重 3-1 多路复用器。

3．分析模型机的功能，设计移位逻辑。

4．分析模型机的工作原理，设计模型机控制信号产生逻辑。

二、实验内容

1．用 VERILOG 语言设计模型机的 8 重 3-1 多路复用器；

2．用 VERILOG 语言设计模型机的移位模块；

3．用 VERILOG 语言设计模型机的控制信号产生逻辑。

$[A]_{补}$=2*4-A

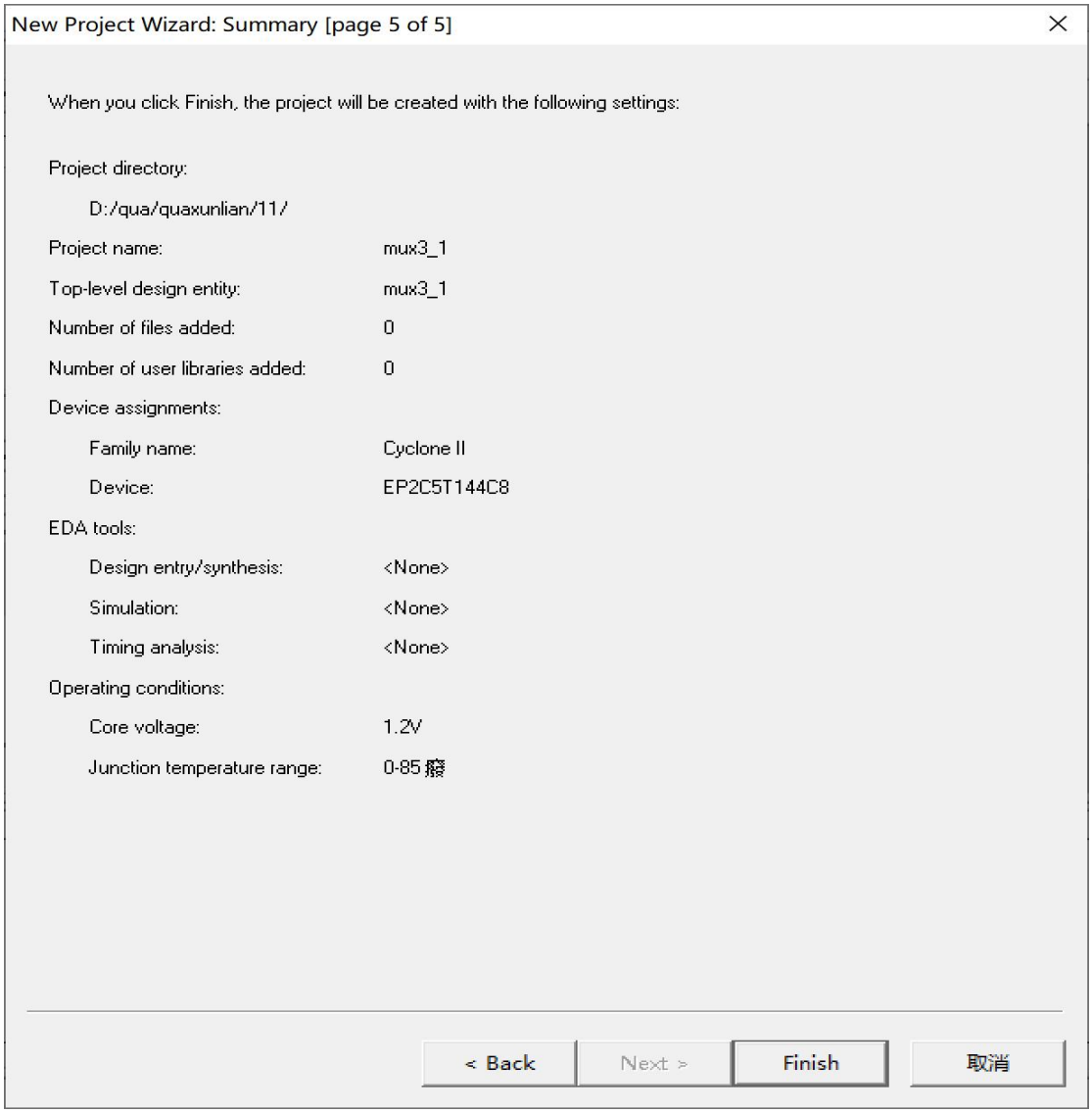
三、实验过程

1、8 重 3-1 多路复用器

A）创建工程（选择的芯片为 family=Cyclone II；name=EP2C5T144C8）

步骤：【File】->【new project wizard】->【next】->【next】->【properties】->【next】->选择芯片类型 family=Cyclone II，name= EP2C5T144C8->【next】->【finish】完成工程创建。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com



B) 编写源代码

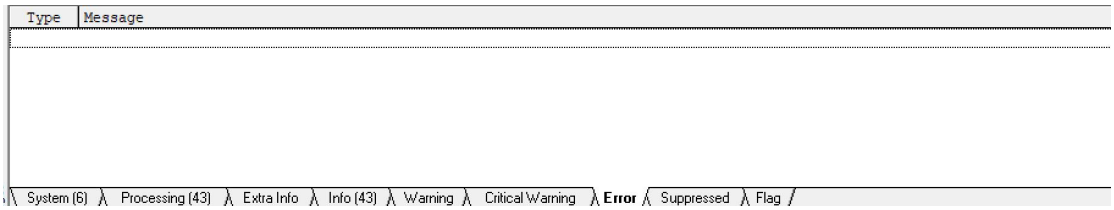
【file】->【Verilog HDL】->写好源代码，保存文件

```
1  module mux3_1(a,b,c,madd,y);
2    input [7:0] a,b,c;
3    input [1:0] madd;
4    output reg[7:0] y;
5
6    always @(*) begin
7         if(madd==2'b00) y=a;
8         else if(madd==2'b01) y=b;
9         else if(madd==2'b10) y=c;
10        else y=8'hZZ;
11        end
12 endmodule
```

PDF-XChange Editor
Click to BUY NOW!
www.tracker-software.com

C) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

无错误无警告

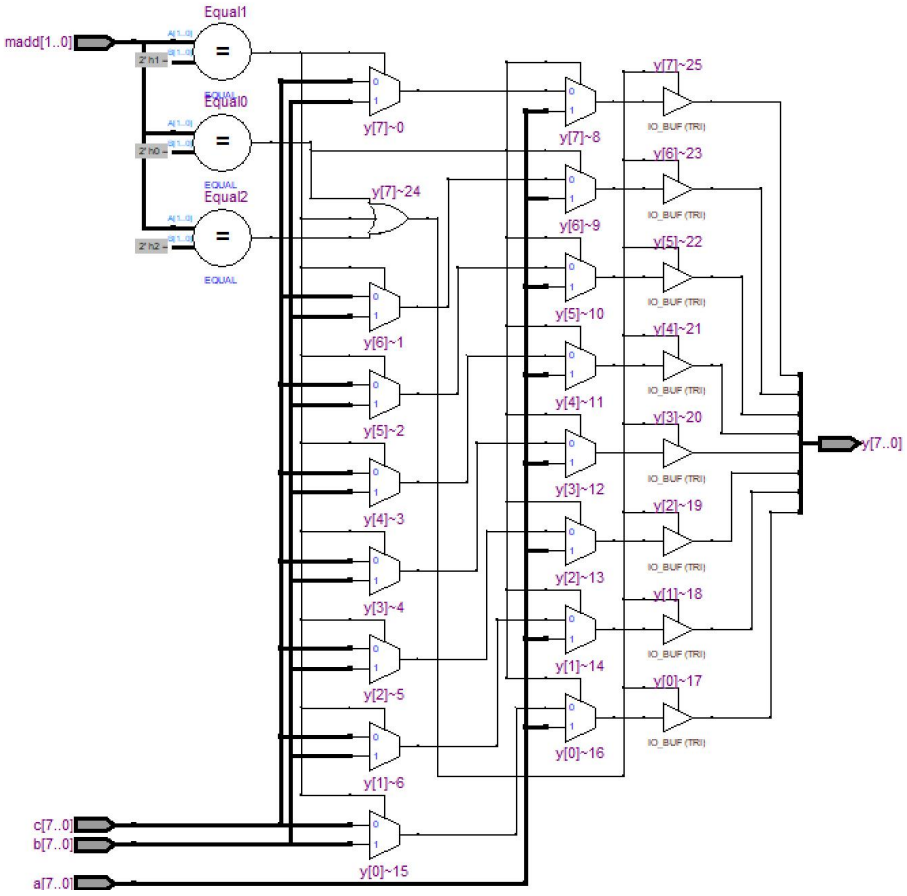


资源消耗：

Flow Summary

Flow Status	Successful - Thu Dec 16 11:12:39 2021
Quartus II Version	9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name	mux3_1
Top-level Entity Name	mux3_1
Family	FLEX10K
Device	EPF10K10TI144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	17 / 576 (3 %)
Total pins	34 / 102 (33 %)
Total memory bits	0 / 6,144 (0 %)

D) RTL 视图







视图分析及结论：

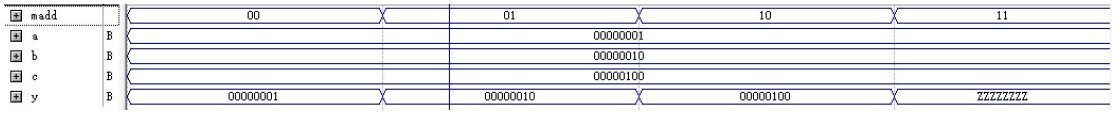
分析：由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的 2-1 选择器构成，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 madd 和 a，b，c，输出信号为 y。各个输出端口之间通过导线相连。

结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

E)功能仿真波形

步骤：新建一个 vector waveform file。通过操作：右击 -【insert】-【insert node or bus】-【node finder】（pins=all；【list】）-【>>】-【ok】-【ok】，在【assignments】-【setting】-【simulator settings】-【simulation mode】中，将 timing 修改为 functional，然后再点击【processing】-【Generate Functional Simulation Netlist】，接着设置输入波形，成功后点击【start simulation】开始仿真，查看输出功能仿真波形图。

功能仿真波形图：



结果分析及结论：

分析：

功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。

madd=00 时，控制输出 y 等于 a，正确

madd=01 时，控制输出 y 等于 b，正确

Madd=10 时，控制输出 y 等于 c，正确

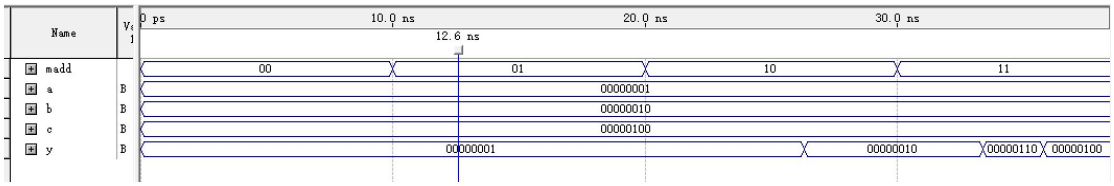
Madd=11 时，控制输出高阻态，正确

结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

F) 时序仿真波形

步骤：新建一个 vector waveform file。通过操作：右击 -【insert】-【insert node or bus】-【node finder】（pins=all；【list】）-【>>】-【ok】-【ok】。接着设置输入波形，然后点击【start simulation】开始仿真，查看时序仿真输出波形图。

时序仿真图：



结果分析及结论：

分析：时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。

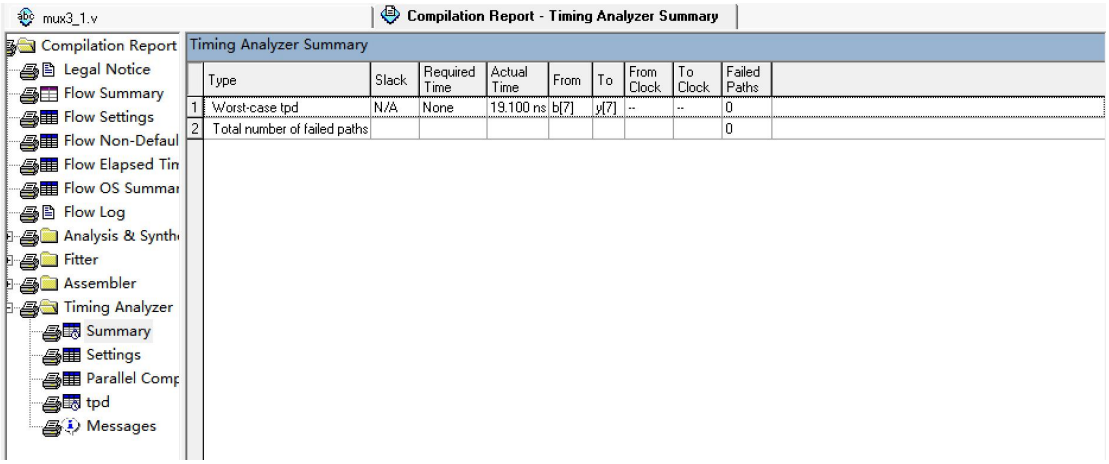
结论：时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

G) 时序分析

操作方法是：编译后，在 compilation report 中选择【timing analysis】-【summary】

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

和【tpd】



Timing Analyzer Summmary 图

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	19.100 ns	b[7]	y[7]
2	N/A	None	18.700 ns	a[7]	y[7]
3	N/A	None	18.300 ns	b[6]	y[6]
4	N/A	None	18.300 ns	a[5]	y[5]
5	N/A	None	18.300 ns	b[4]	y[4]
6	N/A	None	18.300 ns	a[4]	y[4]
7	N/A	None	18.200 ns	b[3]	y[3]
8	N/A	None	18.200 ns	a[3]	y[3]
9	N/A	None	18.200 ns	b[2]	y[2]
10	N/A	None	18.200 ns	a[2]	y[2]
11	N/A	None	18.100 ns	a[1]	y[1]
12	N/A	None	17.900 ns	madd[1]	y[7]
13	N/A	None	17.900 ns	madd[1]	y[6]
14	N/A	None	17.900 ns	a[6]	y[6]
15	N/A	None	17.900 ns	madd[1]	y[5]
16	N/A	None	17.900 ns	b[5]	y[5]
17	N/A	None	17.900 ns	madd[1]	y[4]
18	N/A	None	17.900 ns	madd[1]	y[3]
19	N/A	None	17.900 ns	madd[1]	y[2]
20	N/A	None	17.900 ns	madd[1]	y[1]
21	N/A	None	17.900 ns	madd[1]	y[0]

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
21	N/A	None	17.900 ns	madd[1]	y[0]
22	N/A	None	17.400 ns	madd[0]	y[7]
23	N/A	None	17.400 ns	madd[0]	y[6]
24	N/A	None	17.400 ns	madd[0]	y[5]
25	N/A	None	17.400 ns	madd[0]	y[4]
26	N/A	None	17.400 ns	madd[0]	y[3]
27	N/A	None	17.400 ns	madd[0]	y[2]
28	N/A	None	17.400 ns	madd[0]	y[1]
29	N/A	None	17.400 ns	madd[0]	y[0]
30	N/A	None	16.300 ns	b[1]	y[1]
31	N/A	None	16.300 ns	b[0]	y[0]
32	N/A	None	16.300 ns	a[0]	y[0]
33	N/A	None	15.700 ns	c[7]	y[7]
34	N/A	None	14.900 ns	c[6]	y[6]
35	N/A	None	14.900 ns	c[5]	y[5]
36	N/A	None	14.800 ns	c[3]	y[3]
37	N/A	None	14.800 ns	c[2]	y[2]
38	N/A	None	14.700 ns	c[1]	y[1]
39	N/A	None	14.500 ns	c[4]	y[4]
40	N/A	None	12.900 ns	c[0]	y[0]

Tpd 图

结果分析及结论：

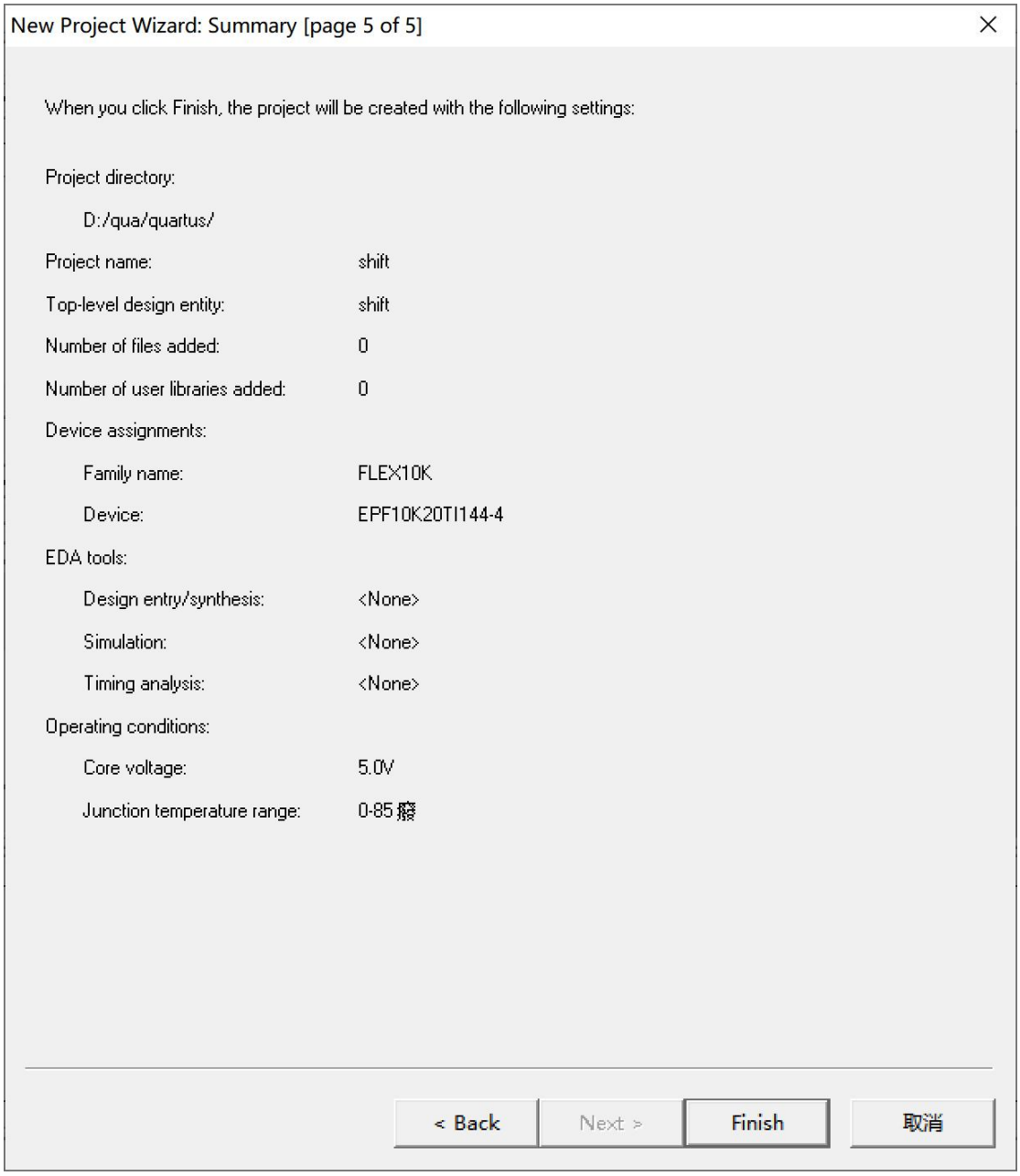
分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 b[7]到 y[7]的最坏定时情况的 tpd 为 19.100ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 a[7]到 y[7] 的 tpd 为 18.700ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

PDF-XChange Editor
Click to BUY NOW!
www.tracker-software.com

2、移位逻辑

A) 创建工程（选择的芯片为 family=FLEX10K；name=EPF10K20TI144-4)



PDF-XChange Editor
Click to BUY NOW!
www.tracker-software.com

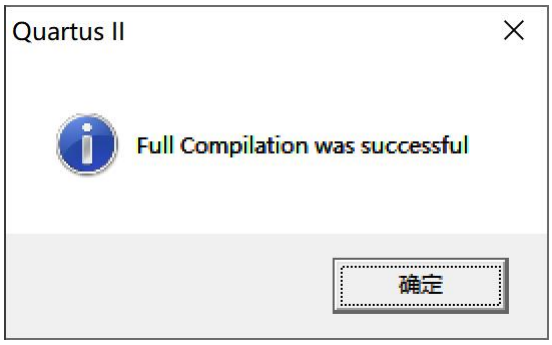
B）编写源代码

shift.v

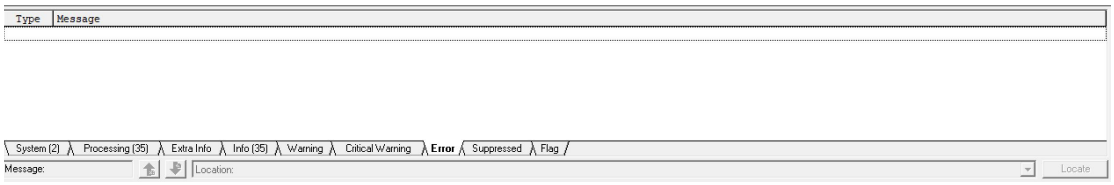
```
1  module shift(fbus,flbus,frbus,a,w,cf);
2      input wire fbus,flbus,frbus;
3      input wire[7:0] a;
4      output reg[7:0] w;
5      output reg cf;
6
7      always @(*) begin
8        cf=0;
9        if(fbus==1'b1&&flbus==1'b0&&frbus==1'b0) w=a;
10       else if(fbus==1'b0&&flbus==1'b1&&frbus==1'b0) begin
11       w[7:1]=a[6:0];
12       w[0]=a[7];
13       cf=a[7];
14       end
15
16       else if(fbus==1'b0&&flbus==1'b0&&frbus==1'b1) begin
17       w[6:0]=a[7:1];
18       w[7]=a[0];
19       cf=a[0];
20       end
21
22       else if(fbus==1'b0&&flbus==1'b0&&frbus==1'b0) begin
23       w=8'hZZ;
24       cf=0;
25       end
26
27       else w=8'hZZ;
28     end
29
30 endmodule
```

C）编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

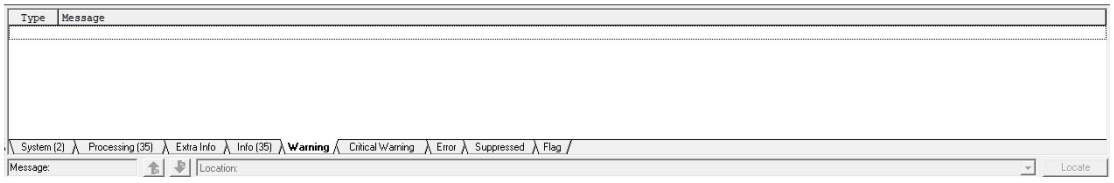
编译成功



无错误



无警告信息





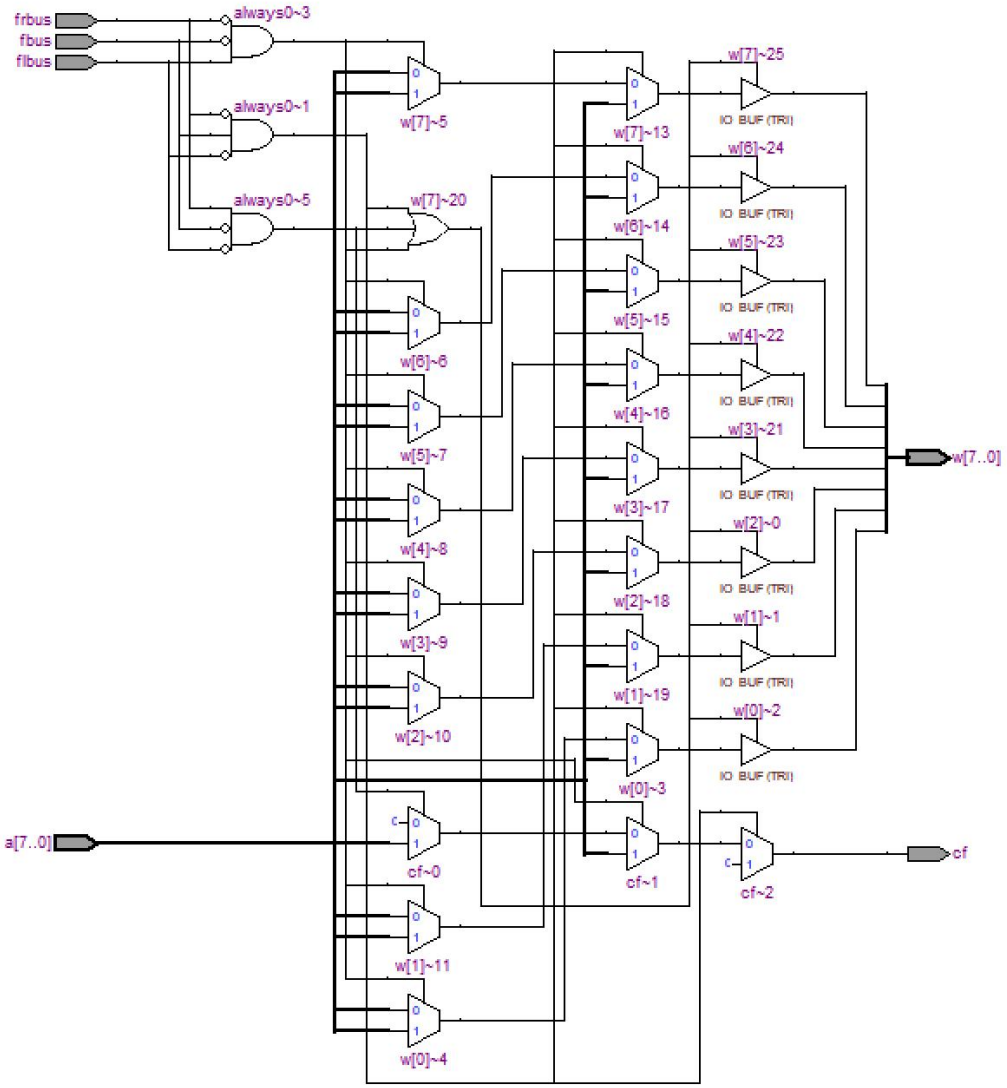


资源消耗

Flow Summary

Flow Status	Successful - Fri Dec 17 15:44:00 2021
Quartus II Version	9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name	shift
Top-level Entity Name	shift
Family	FLEX10K
Device	EPF10K10TI144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	21 / 576 (4 %)
Total pins	20 / 102 (20 %)
Total memory bits	0 / 6,144 (0 %)

D) RTL 视图

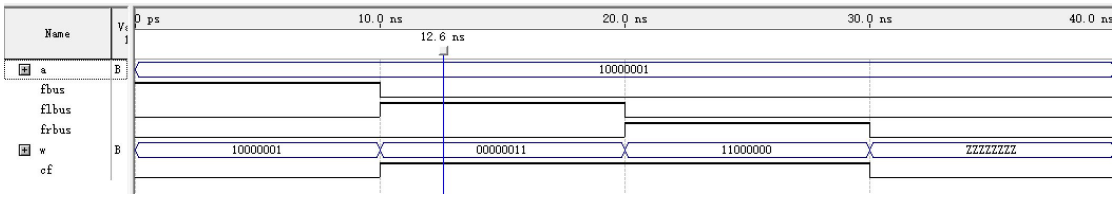


PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

结果分析及结论：

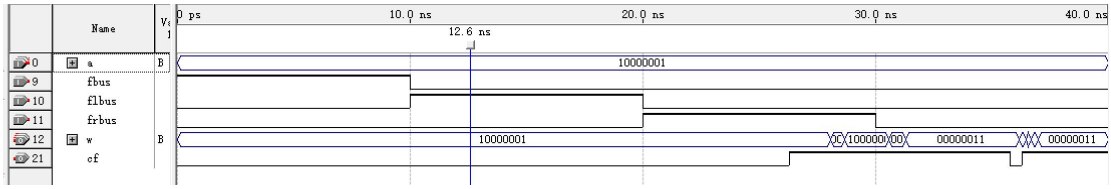
分析：由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较器：当输入相等时输出 1，不相等时输出 0；还有大部分的 2-1 选择器构成，当控制信号为 0 时，输出第一位，控制信号为 1 时，输出第二位。图中输入信号为 fbus，frbus，flbus 和 a，输出信号为 w。各个输出端口之间通过导线相连。
结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

E）功能仿真波形



结果分析及结论：
分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。
当 fbus=1，frbus=0，flbus=0，不执行移位操作，输出等于输入，cf 不改变
当 fbus=0，frbus=1，flbus=0，执行右移，输出等于输入右移移位，有进位的话 cf 为 1
当 fbus=0，frbus=0，flbus=1，执行左移，输出等于输入左移一位，cf 不改变
当控制信号全为 0 时，输出为高阻态，正确
结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

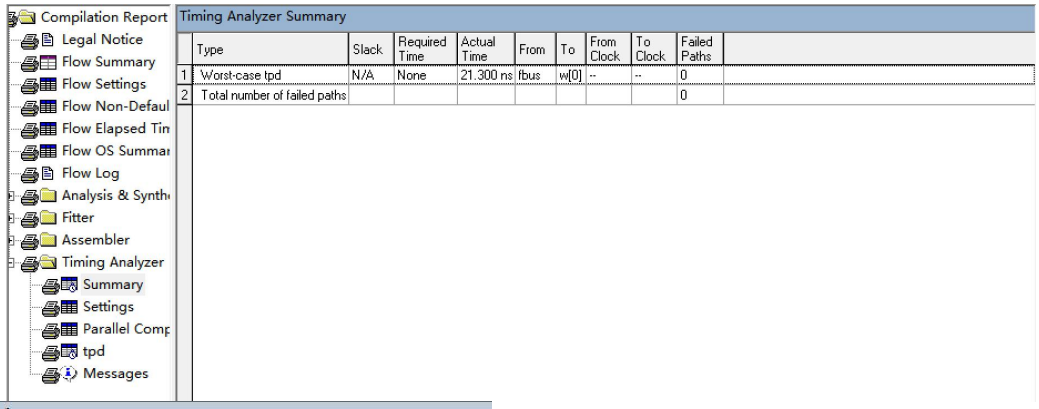
F）时序仿真波形



结果分析及结论：
分析：时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。
结论：时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

G) 时序分析



tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	21.300 ns	flbus	w[7]
2	N/A	None	21.300 ns	fbus	w[7]
3	N/A	None	21.300 ns	flbus	w[2]
4	N/A	None	21.300 ns	fbus	w[2]
5	N/A	None	21.300 ns	flbus	w[1]
6	N/A	None	21.300 ns	fbus	w[1]
7	N/A	None	21.300 ns	flbus	w[0]
8	N/A	None	21.300 ns	fbus	w[0]
9	N/A	None	20.800 ns	frbus	w[7]
10	N/A	None	20.800 ns	frbus	w[2]
11	N/A	None	20.800 ns	frbus	w[1]
12	N/A	None	20.800 ns	frbus	w[0]
13	N/A	None	20.600 ns	flbus	w[6]
14	N/A	None	20.600 ns	fbus	w[6]
15	N/A	None	20.600 ns	flbus	w[5]
16	N/A	None	20.600 ns	fbus	w[5]
17	N/A	None	20.600 ns	flbus	w[4]
18	N/A	None	20.600 ns	fbus	w[4]
19	N/A	None	20.600 ns	flbus	w[3]
20	N/A	None	20.600 ns	fbus	w[3]
21	N/A	None	20.100 ns	frbus	w[6]
22	N/A	None	20.100 ns	frbus	w[5]
23	N/A	None	20.100 ns	frbus	w[4]
24	N/A	None	20.100 ns	frbus	w[3]
25	N/A	None	19.300 ns	a[6]	w[7]
26	N/A	None	19.300 ns	a[3]	w[2]
27	N/A	None	19.000 ns	a[1]	w[2]

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
33	N/A	None	18.300 ns	a[5]	w[4]
34	N/A	None	18.300 ns	a[4]	w[3]
35	N/A	None	17.500 ns	a[7]	w[0]
36	N/A	None	17.400 ns	a[2]	w[1]
37	N/A	None	17.300 ns	a[0]	w[7]
38	N/A	None	17.300 ns	a[0]	w[1]
39	N/A	None	16.900 ns	a[7]	w[6]
40	N/A	None	16.700 ns	a[2]	w[3]
41	N/A	None	16.400 ns	a[0]	cf
42	N/A	None	16.300 ns	frbus	cf
43	N/A	None	16.300 ns	flbus	cf
44	N/A	None	16.300 ns	fbus	cf
45	N/A	None	16.100 ns	a[1]	w[1]
46	N/A	None	15.700 ns	a[6]	w[6]
47	N/A	None	15.700 ns	a[3]	w[3]
47	N/A	None	15.700 ns	a[3]	w[3]
48	N/A	None	15.400 ns	a[5]	w[5]
49	N/A	None	15.400 ns	a[4]	w[4]
50	N/A	None	14.600 ns	a[7]	w[7]
51	N/A	None	14.500 ns	a[2]	w[2]
52	N/A	None	14.400 ns	a[0]	w[0]
53	N/A	None	13.100 ns	a[7]	cf

结果分析及结论：

分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 fbus 到 w[0] 的最坏定时情况的 tpd 为 21.300ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 fbus 到 w[7] 的 tpd 为 21.300ns。

结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

3、控制信号产生逻辑

A) 创建工程（选择的芯片为 family=FLEX10K；name=EPF10K20TI144-4)



PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

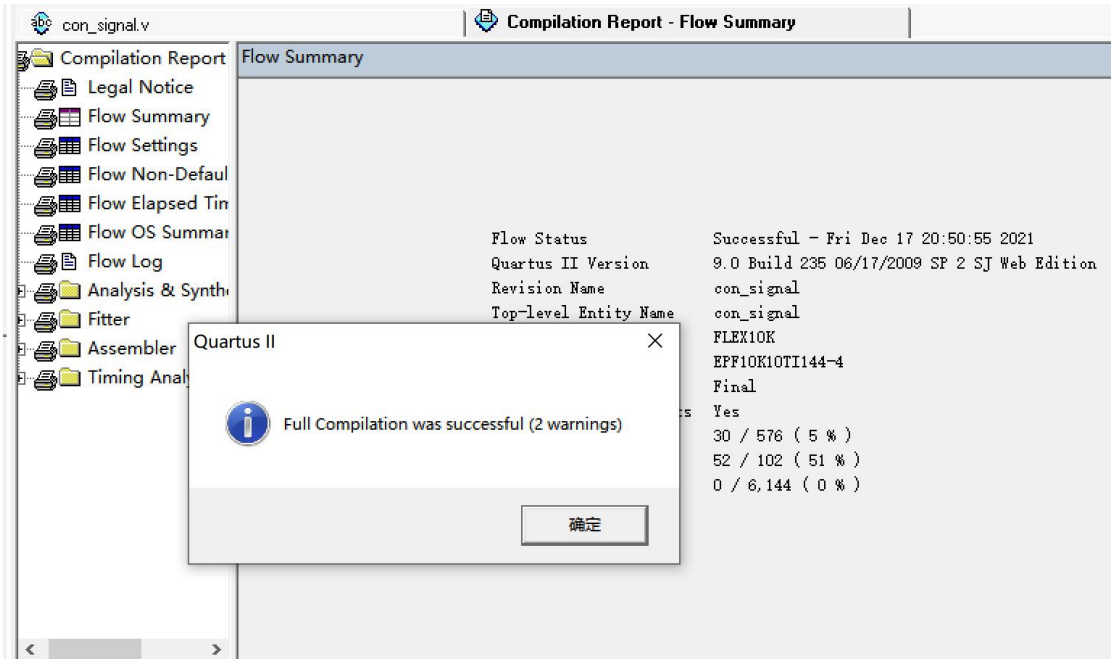
B）编写源代码

con_signal.v

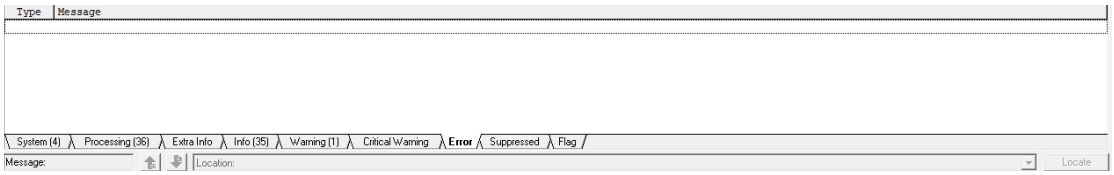
```
1  module con_signal(
2      input mova,movb,movc,add,sub,andl,notl,rsr,rsl,jmp,jz,
3            jc,inl,outl,nop,halt,sm,c,z,
4      input [7:0] ir,
5      output reg[3:0] alu_s,
6      output reg[1:0] madd,reg_ra,reg_wa,
7      output reg pc_ld,pc_inc,reg_we,ram_xl,ram_dl,alu_m,shi_fbus,
8                 shi_flbus,shi_frbus,ir_ld,cf_en,zf_en,sm_en,in_en,out_en);
9       always @(*) begin
10         sm_en=~halt;
11         ir_ld=~sm;
12         ram_dl=(~sm)|movc|jmp|(jz&z)|(jc&c);
13         ram_xl=movb;
14         shi_fbus=mova|movb|add|sub|andl|notl|outl;
15         shi_flbus=rsl;
16         shi_frbus=rsr;
17         alu_s=ir[7:4];
18         alu_m=andl|notl|add|sub|rsr|rsl|outl;
19         cf_en=add|sub|rsr|rsl;
20         zf_en=add|sub;
21         pc_ld=jmp|(jc&c)|(jz&z);
22         pc_inc=(~sm)|(jc&(~c))|(jz&(~z));
23         reg_we=~(sm&(mova|movc|add|sub|andl|notl|rsr|rsl|inl));
24         in_en=inl;
25         out_en=outl;
26         reg_ra=ir[1:0];
27         reg_wa=ir[3:2];
28         if(sm==1&&movc==1) madd=01;
29         else if(sm==1&&movb==1) madd=2'b10;
30         else madd=00;
31         end
32 endmodule
```

C）编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

编译成功

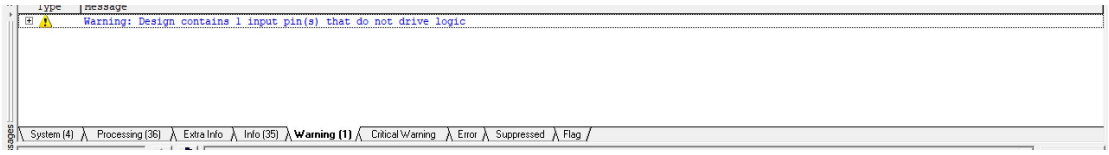


无错误

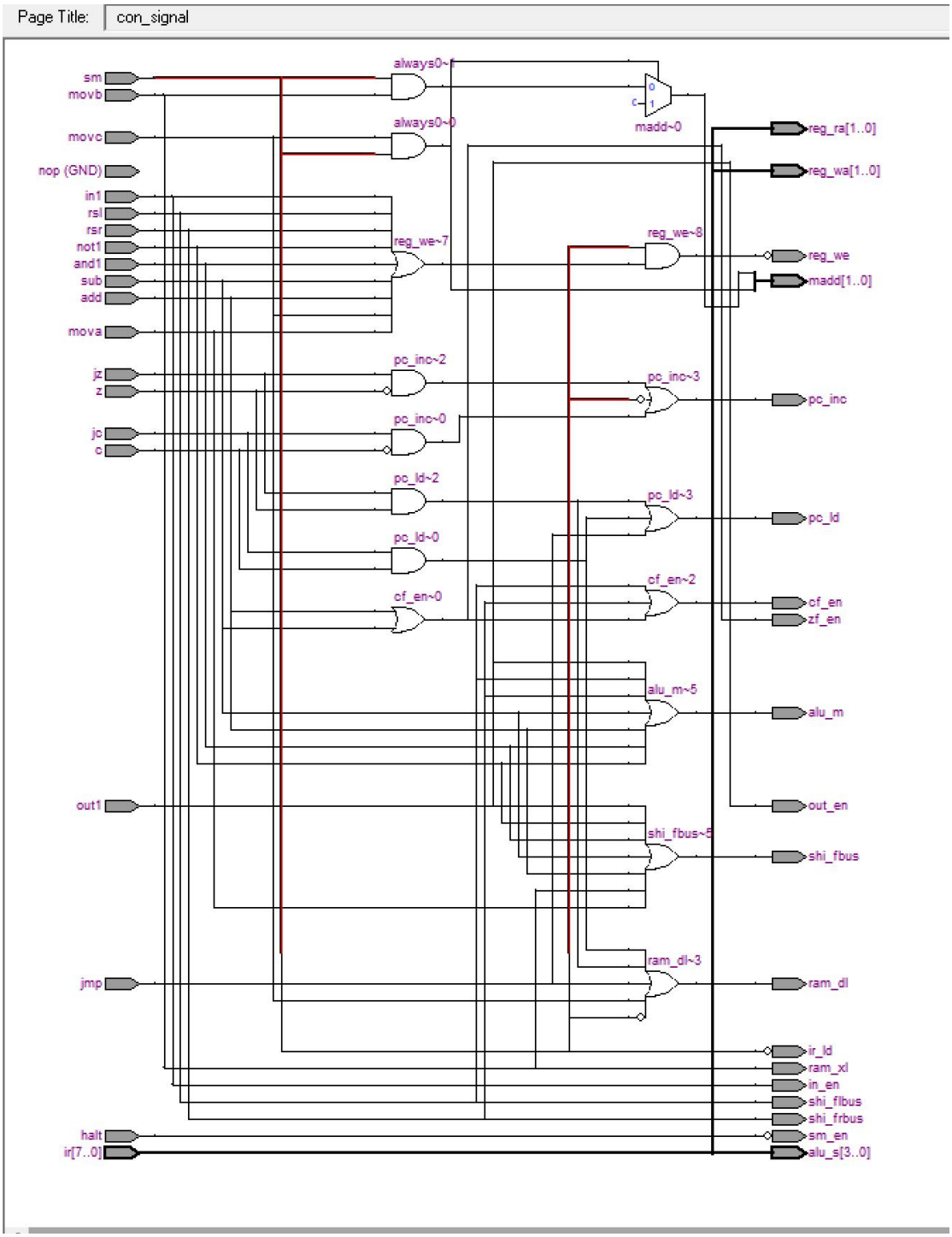


PDF-XChange Editor
Click to BUY NOW!
www.tracker-software.com

警告信息



D) RTL 视图



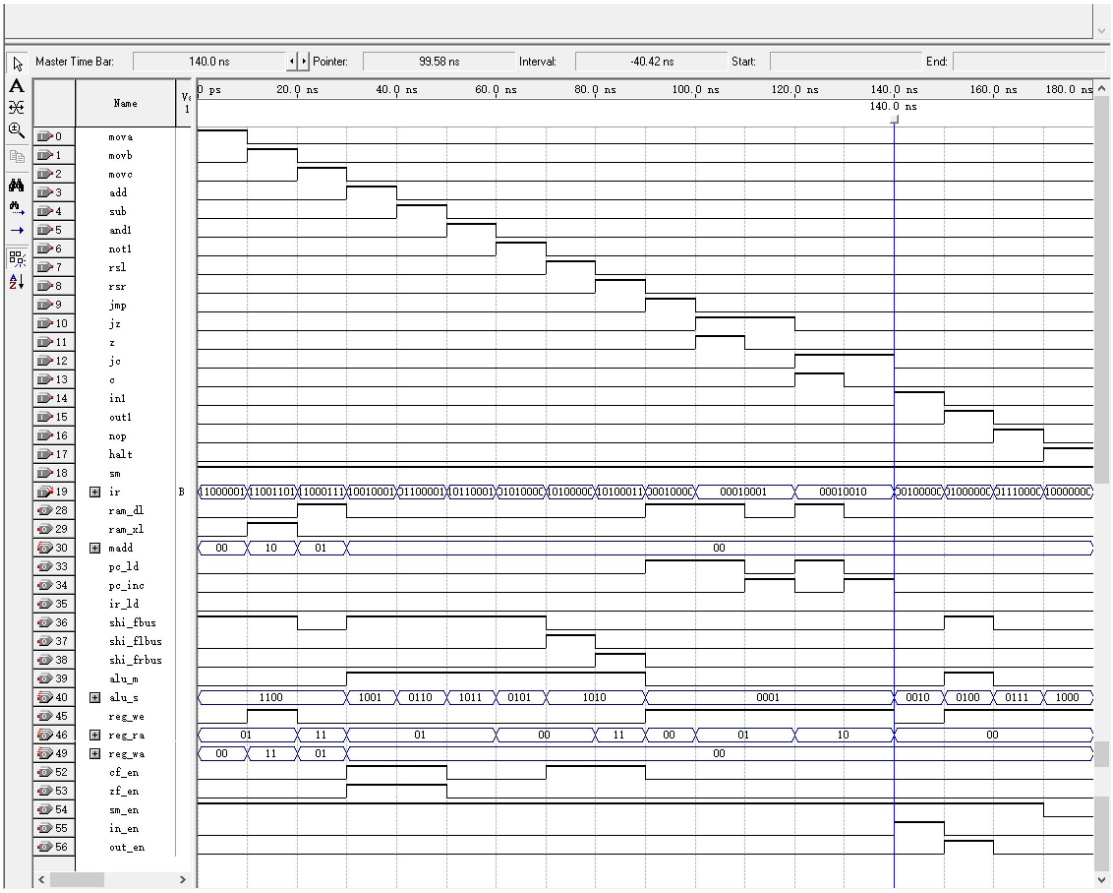
结果分析及结论：

分析：由视图可得，视图左边为输入，右边为输出。其中连接有一系列的元器件。比如比较

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

器：当输入相等时输出 1，不相等时输出 0；还有大部分的与或门。图中输入信号为 sm 等 20 个，输出信号包括 reg_ra 等 19 种情况。各个输出端口之间通过导线相连。
结论：一个功能的实现需要经过多重门的处理后才能实现，一个元件的内部原理结构图十分复杂。

E) 功能仿真波形

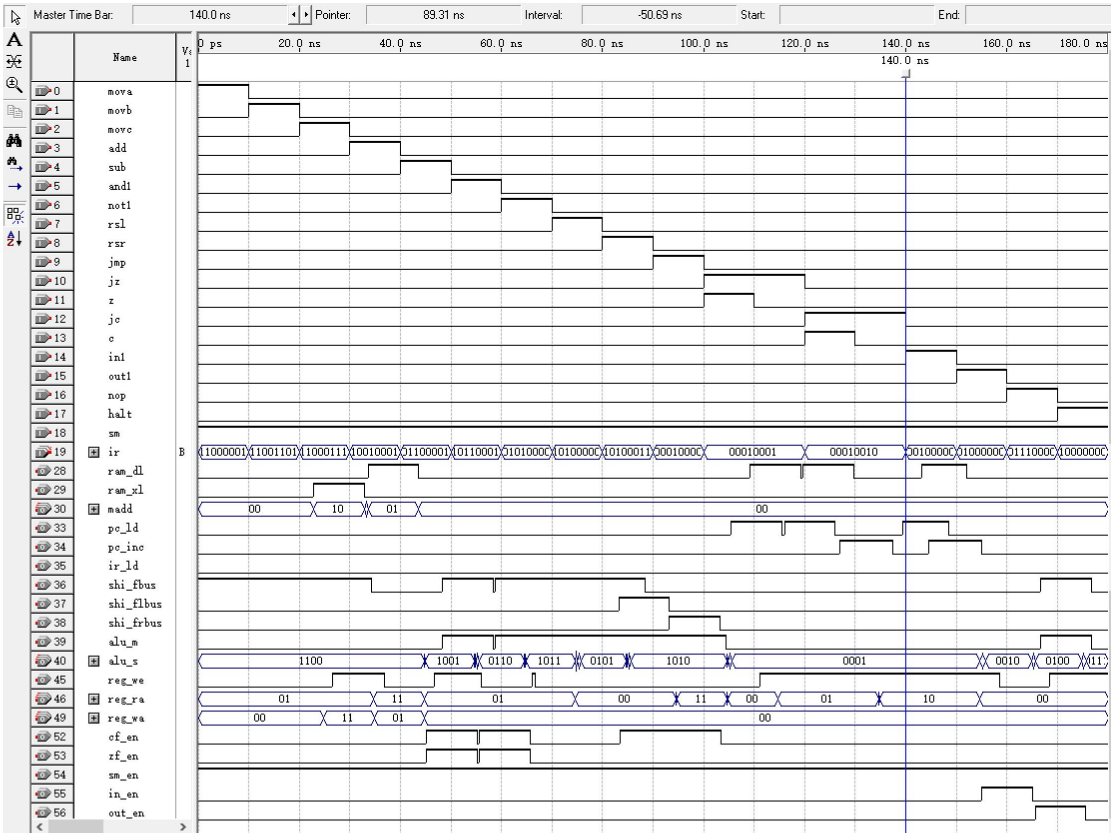


结果分析及结论：
分析：功能仿真是指不考虑器件延时和布线延时的理想情况下对源代码进行逻辑功能的验证。由仿真波形可得，对于输入状态的变化，输出结果实时变化，没有延迟，其结果与电路设计的真值表的结果相对应。
（1）当 mova 指令执行时，shi_fbus 和 sm_en 输出 1，其他输出为 0，madd 输出 00，alu_s 输出为 1100，reg_ra 输出 01，reg_wa 输出 00，正确
（2）当 movb 指令执行时，ram_xl 和 shi_fbus 和 reg_we 和 sm_en 输出为 1，其他输出为 0，madd 输出为 10，alu_s 输出为 1100，reg_ra 输出 01，reg_wa 输出 11，正确
（3）当 movc 指令执行时，ram_dl 和 sm_en 输出为 1，其他输出为 0，madd 输出 01，alu_s 输出 1100，reg_ra 输出 11，reg_wa 输出 01，正确
（4）当 add 指令执行时，shi_fbus，alu_en，cf_en，zf_en，sm_en 输出为 1，其他输出为 0，alu_s 为 1001，reg_ra 输出 01，reg_wa 输出 00，正确
（5）当 sub 指令执行时，shi_fbus 和 alu_m，cf_en，zf_en 和 sm_en 输出为 1，其他输出为 0，alu_s 输出 0110，reg_ra 输出 01，reg_wa 输出 00，正确
（6）当 and1 指令执行时，shi_fbus 和 alu_m 和 sm_en 输出 1，其他输出 0，alu_s 输出 1011，reg_ra 输出 01，reg_wa 输出 00，正确
（7）not1 指令执行时，shi_fbus 和 alu_m 和 sm_en 输出 1，其他输出 0，alu_s 输出 0101，reg_ra 输出 00，reg_wa 输出 00，正确
（8）rsl 指令执行时，shi_flbus 和 alu_m 和 cf_en 和 sm_en 输出 0，其他输出 0，alu_s 输出 1010，reg_ra 和 reg_wa 输出 00，正确
（9）rsr 指令执行时，shi_frbus 和 alu_m 和 cf_en 和 sm_en 输出 1，其他输出 0，alu_s 输出 1010，reg_ra 输出 11，reg_wa 输出 00，正确

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

（10）jmp 指令执行时，ram_dl，pc_ld，reg_we 和 sm_en 输出 1，其他输出 0，alu_s 输出 0001，reg_ra 和 reg_wa 输出 00，正确
（11)jz 指令为 1 和 jc 指令为 1 时,若 z 和 c 为 1 时,ram_dl 和 pc_ld 和 reg_we 和 sm_en 输出为 1，其他输出为 0，正确
若 z 和 c 为 0 时，pc_inc 和 reg_we 和 sm_en 输出 1，其他输出 0，正确
（12）in1 指令执行时，sm_en 和 in_en 输出 1，其他输出 0，正确
（13）out1 指令执行时，sm_en 和 out_en 输出 1，其他输出 0，正确
（14）nop 指令执行时，sm_en 输出 1，其他输出 0，正确
（15）halt 指令执行时，输出全为 0，正确
结论：功能仿真操作简单，能体现和验证实验的功能，但忽略延迟的影响会使结果与实际结果有一定误差。

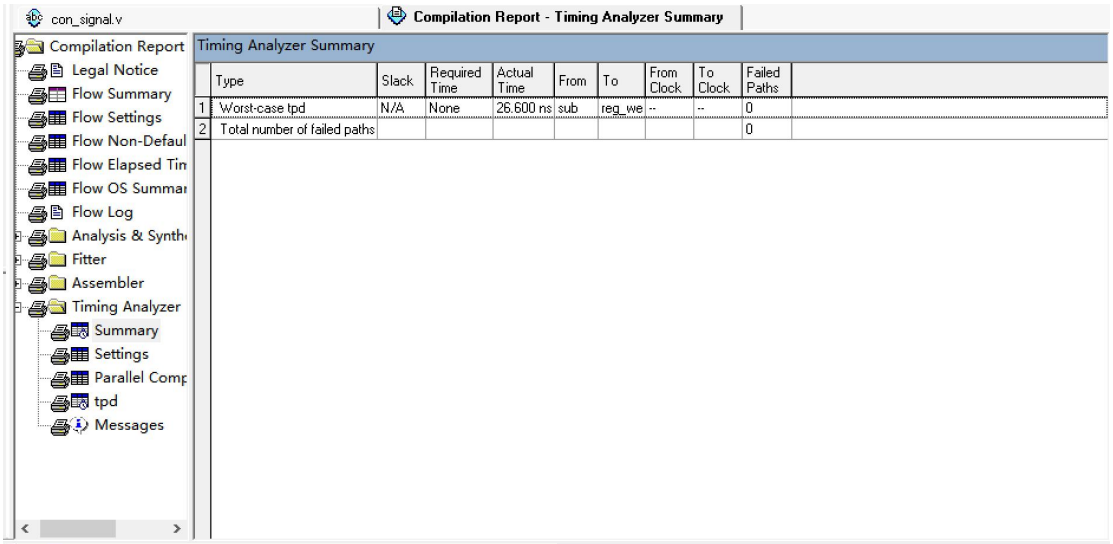
F）时序仿真波形



结果分析及结论：
分析：时序仿真是指在布线后进行，是最接近真实器件运行的仿真，它与特定的器件有关，又包含了器件和布线的延时信息。由波形可得，当输入状态发生改变时，输出结果并未同时改变，而是有一定延迟，同时由于输入状态的改变，导致电路出现“冒险”，导致输出结果并未与预期结果相同。
结论：时序仿真可以用来验证程序在目标器件中的时序关系。同时考虑了器件的延迟后，其输出结果跟接近实际情况，但是考虑的情况过多，不容易操作，容易产生错误。时序仿真不仅反应出输出和输入的逻辑关系，同时还计算了时间的延时信息，是与实际系统更接近的一种仿真结果。不过，要注意的是，这个时间延时是仿真软件“估算”出来的。

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

G) 时序分析



tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	26.600 ns	sub	reg_we
2	N/A	None	26.300 ns	and1	reg_we
3	N/A	None	26.300 ns	not1	reg_we
4	N/A	None	26.100 ns	add	reg_we
5	N/A	None	23.100 ns	jc	ram_dl
6	N/A	None	22.100 ns	c	ram_dl
7	N/A	None	21.700 ns	rsl	reg_we
8	N/A	None	21.200 ns	rsr	reg_we
9	N/A	None	19.700 ns	jz	ram_dl
10	N/A	None	19.700 ns	z	ram_dl
11	N/A	None	19.400 ns	jc	pc_ld
12	N/A	None	19.200 ns	jmp	ram_dl
13	N/A	None	18.800 ns	sub	shi_fbus
14	N/A	None	18.800 ns	sub	alu_m
15	N/A	None	18.500 ns	and1	shi_fbus
16	N/A	None	18.500 ns	not1	shi_fbus
17	N/A	None	18.500 ns	and1	alu_m
15	N/A	None	18.500 ns	and1	shi_fbus
16	N/A	None	18.500 ns	not1	shi_fbus
17	N/A	None	18.500 ns	and1	alu_m
18	N/A	None	18.500 ns	not1	alu_m
19	N/A	None	18.500 ns	in1	reg_we
20	N/A	None	18.400 ns	c	pc_ld
21	N/A	None	18.300 ns	add	shi_fbus
22	N/A	None	18.300 ns	add	alu_m
23	N/A	None	17.400 ns	jz	pc_inc
24	N/A	None	16.900 ns	z	pc_inc
25	N/A	None	16.800 ns	movc	reg_we
26	N/A	None	16.700 ns	out1	shi_fbus
27	N/A	None	16.700 ns	out1	alu_m
28	N/A	None	16.600 ns	mova	reg_we
29	N/A	None	16.000 ns	jz	pc_ld
30	N/A	None	16.000 ns	z	pc_ld
28	N/A	None	16.600 ns	mova	reg_we
29	N/A	None	16.000 ns	jz	pc_ld
30	N/A	None	16.000 ns	z	pc_ld

tpd

	Slack	Required P2P Time	Actual P2P Time	From	To
34	N/A	None	15.600 ns	out1	out_en
35	N/A	None	15.500 ns	jmp	pc_ld
36	N/A	None	15.200 ns	add	zf_en
37	N/A	None	15.200 ns	add	cf_en
38	N/A	None	15.100 ns	in1	in_en
39	N/A	None	15.000 ns	jc	pc_inc
40	N/A	None	14.900 ns	halt	sm_en
41	N/A	None	14.900 ns	ir[6]	alu_s[2]
42	N/A	None	14.800 ns	ir[3]	reg_wa[1]
43	N/A	None	14.800 ns	ir[2]	reg_wa[0]
44	N/A	None	14.700 ns	ir[1]	reg_ra[1]
45	N/A	None	14.700 ns	ir[4]	alu_s[0]
46	N/A	None	14.600 ns	sm	reg_we
47	N/A	None	14.600 ns	ir[0]	reg_ra[0]
48	N/A	None	14.600 ns	ir[7]	alu_s[3]
49	N/A	None	14.500 ns	c	pc_inc
50	N/A	None	14.400 ns	rsl	alu_m
51	N/A	None	14.400 ns	rsr	alu_m
52	N/A	None	14.200 ns	movb	shi_fbus
53	N/A	None	14.200 ns	mova	shi_fbus
54	N/A	None	13.800 ns	sm	ram_dl
55	N/A	None	13.600 ns	movc	ram_dl
56	N/A	None	13.600 ns	movc	madd[0]
57	N/A	None	13.500 ns	rsl	cf_en
58	N/A	None	13.500 ns	rsr	cf_en
59	N/A	None	13.500 ns	sm	ir_ld
60	N/A	None	13.400 ns	sm	madd[0]
61	N/A	None	13.200 ns	rsr	shi_frbus
62	N/A	None	13.200 ns	rsl	shi_flbus
63	N/A	None	13.100 ns	movc	madd[1]
64	N/A	None	12.900 ns	movb	ram_xl
65	N/A	None	12.900 ns	movb	madd[1]
66	N/A	None	12.800 ns	sm	madd[1]
67	N/A	None	12.200 ns	sm	pc_inc

PDF-XChange Editor Click to BUY NOW! www.tracker-software.com
PDF-XChange Editor Click to BUY NOW! www.tracker-software.com

结果分析及结论：
分析：由图可得，Timing Analyzer Summmary 总结所有经典定时分析的结果，并报告每个定时特性的最坏情况定时。比如从 sub 到 reg_we 的最坏定时情况的 tpd 为 26.600ns。下面的 tpd 报告表则给出了源节点和目标节点之间的 tpd 延迟时间，比如第二行中 and1 到 reg_we 的 tpd 为 26.300ns。
结论：实际连接图中个元器件连接之间是存在时间延迟的，而且不同的元器件之间的时间延迟也不相同。

四、思考题

1．移位逻辑不工作时，输出应该为何值？为什么？

答：输出应成高阻，防止数据通路发生冲突，产生主线竞争。

2、任选一条指令，介绍指令的过程、信息流动的情况以及执行时控制信号的值。

答：mova 指令：将编码 01（RAA1、RAA0 提供）对应寄存器 B 数据（经 S 口）读出经 ALU（S3～S0 和 M 的控制） 移位逻辑（FL->BUS， FR->BUS 控制） BUS 由/WE 控制将 BUS 上的数据在时钟下降沿写入编码 00（RWBA1、RWBA0 提供）对应的寄存器 A 中。

控制信号：shi_fbus 和 sm_en 为 1，madd 为 00，alu_s 为 1100，reg_ra 输出 01，reg_wa 为 00

3．如何产生正确的控制信号以及具体的编程实现？

答：应当逐个分析每个控制信号在不同的指令下对应的状态，利用逻辑函数进行状态的总和。

五、实验总结、必得体会及建议

1、从需要掌握的理论、遇到的困难、解决的办法以及经验教训等方面进行总结。

答：这次实验我掌握了选择器，移位逻辑，控制器的工作原理如何构造，这次实验让我对模型机的工作原理有了更加深入的了解，同时在运行调试过程中的锁存器问题也让我对组合电路以及时序电路的区别与表现理解更为细致。在编写模型机各个部件时也应当具有全局观念，要考虑把这个部件最终组合后可能的运行情况，使得设计的差错尽可能在最初的时候就能避免。还有比如不懂使用 QuartusII 进行波形仿真。 但通过上网查询相关资料和询问同学后得以解决问题，并通过分析报告发现电路中的问题。

2、对本实验内容、过程和方法的改进建议（可选项）。

最好能给出控制器各个控制信号的输出功能表